# Max Profit Finder

Category: Divide and Conquer | Language: Python 3

## 1. Aim

The goal is to find the best day to buy and the best later day to sell using a list of prices. The program compares a Divide and Conquer solution with a simpler linear scan.

## 2. Algorithm

First, the program calculates the difference between each pair of consecutive prices. The sum of differences between two dates equals the profit over that interval.

To find the maximum sum, **max_subarray** divides the differences into left and right halves. It finds the best subarray on the left, on the right, and one that crosses the middle. The highest result is returned. If the maximum profit is not positive, the program reports no profitable trade.

## 3. Example

| Day | Price |
|---|---|
| Day 1 | 10 |
| Day 2 | 7 |
| Day 3 | 5 |
| Day 4 | 8 |
| Day 5 | 11 |
| Day 6 | 9 |
| Day 7 | 12 |
| Day 8 | 10 |

**Output:** Buy on Day 3 at $5.00; sell on Day 7 at $12.00; maximum profit = **$7.00**. The linear method also returned $7.00.

## 4. Complexity

The divide-and-conquer maximum-subarray step has recurrence T(n) = 2T(n/2) + O(n), giving **O(n log n)** time. Storing daily differences uses **O(n)** additional space. The linear scan takes **O(n)** time and **O(1)** extra space.

## 5. Testing and conclusion

I tested an increasing/decreasing mix, a fully decreasing sequence, a one-day list, and 100 randomly generated price lists. The divide-and-conquer and linear answers agreed on maximum profit in the tests.

Conclusion: Divide and Conquer is an interesting way to solve this problem, even though the linear scan is more efficient for this particular task. The dataset is only a learning example.